

Efficient Adaptation of Small Language Models for Reliable Mathematical Reasoning

Kanak Anil Malpani

Department of Electrical and Electronics Engineering

National Institute of Technology Tiruchirappalli

Tiruchirappalli, India

mrkanakmalpani@gmail.com

Abstract

Efficient training and adaptation are central to deploying language models under resource constraints. We present a controlled comparison of six parameter-efficient fine-tuning (PEFT) methods—LoRA, QLoRA, DoRA, PiSSA, LoftQ, and AdaLoRA—across the full cross-product of three base models (Qwen2.5-1.5B, Qwen2.5-7B, Llama-3-8B) and two mathematical reasoning benchmarks (GSM8K, MATH), with every configuration trained on Meta-MathQA under one identical budget and reported as mean $\pm$ standard deviation over three seeds. Qwen2.5-1.5B + PiSSA reaches $71.40 \pm 0.27\%$ on GSM8K, exceeding the unadapted 7B base model ($68.50 \pm 0.42\%$; Welch’s $t(3.4) = 10.06$, $p = 0.001$). Significance testing shows that the method ordering—PiSSA > DoRA > LoRA $\approx$ AdaLoRA $\approx$ LoftQ $\approx$ QLoRA—is *identical* across all three base models (Kendall’s $\tau = 1.0$), but that only the PiSSA and DoRA gains over LoRA survive multiple-comparison correction: the LoRA/QLoRA/LoftQ/AdaLoRA cluster is statistically indistinguishable at $n=3$. Four-bit QLoRA cuts peak training VRAM by up to 69.6% (72.8% with PagedAdamW), within one point of the 68.7% predicted by a static-memory model, while PagedAdamW trades 15–20% throughput for memory safety. We add rank and target-module ablations, a GSM8K error taxonomy, and the finding that PiSSA’s spectral initialization at $r=16$ outperforms LoRA at $r=64$, and we release the training, evaluation, and profiling code with the exact configuration for every run.¹

1 Introduction

Adapting a pre-trained language model by Full Fine-Tuning (FFT) updates every parameter, and the cost of doing so is what keeps most practitioners out. Optimizer states and gradients scale linearly

with parameter count, each downstream task produces its own full-sized checkpoint, and narrow task-specific data tends to overwrite the general reasoning pathways acquired during pre-training.

Parameter-efficient fine-tuning (PEFT) removes most of that cost by freezing the backbone and training a small set of auxiliary parameters (Houlsby et al., 2019). LoRA (Hu et al., 2022) is the dominant instance: it constrains the update to a low-rank factorization, which can be merged back into the base weights so that inference latency is unchanged. QLoRA (Dettmers et al., 2023) adds 4-bit quantization of the frozen backbone, and a family of refinements—DoRA (Liu et al., 2024), PiSSA (Meng et al., 2024), LoftQ (Li et al., 2024), AdaLoRA (Zhang et al., 2023), rsLoRA (Kalamajdziewski, 2023)—each reports gains over LoRA in its own paper, under its own hardware, data, and evaluation protocol.

That is the gap this paper addresses. A practitioner choosing between these methods for a specific deployment—a small model, a specialized task, a consumer GPU—cannot read the answer off six separate papers. We therefore fix the model, data, budget, and evaluation harness, and vary only the adaptation algorithm.

We use *reliable* in a deliberately narrow sense throughout: (i) *measurement* reliability, meaning every accuracy number is a three-seed mean with a reported standard deviation and every method comparison is accompanied by a significance test, and (ii) *failure-mode* reliability, meaning we characterize *how* adapted models fail rather than only how often. We do not claim reliability in the broader sense of robustness to distribution shift or preservation of general capability; Section LIMITATIONS states this explicitly.

Contributions.

1. A complete 3×6 model $\times$ method comparison on GSM8K and MATH under one fixed

¹Track 1: Direct Submission, non-archival.

budget, with three-seed statistics (Table 2).

2. Welch’s t -tests with Holm–Bonferroni correction over all method pairs, separating the differences that survive correction from those that do not (Table 3).
3. Rank, target-module, and scaling ablations, and the finding that PiSSA’s spectral initialization at $r=16$ outperforms LoRA at $r=64$.
4. Consumer-GPU profiling (RTX 4060/4090) of peak VRAM, throughput, and energy, cross-checked against an analytic static-memory model.
5. A GSM8K error taxonomy with a stated annotation protocol, and practitioner guidance for method selection under a VRAM budget.

2 Related Work

PEFT methods. Adapter-style tuning (Houlsby et al., 2019) established that a small number of injected parameters suffices for downstream adaptation; Aghajanyan et al. (2021) gave the complementary argument that fine-tuning updates have low intrinsic dimension. LoRA (Hu et al., 2022) made this practical by removing inference overhead through weight merging. The methods we compare all modify LoRA along one axis: quantization of the frozen base (QLoRA, LoftQ), initialization (PiSSA, LoftQ), parameterization (DoRA), rank allocation (AdaLoRA), or scaling (rsLoRA). Structured pruning of over-provisioned adapters (Yu et al., 2025) is a further direction we do not evaluate here.

PEFT benchmarking. Lialin et al. (2023) survey the design space and note that methods beating LoRA in their own papers often fail to do so under constrained budgets—a finding our significance analysis reproduces quantitatively. Biderman et al. (2024) compare LoRA against FFT on code and math and characterize the learning/forgetting trade-off. Our study is complementary and narrower in a specific way: rather than PEFT-versus-FFT, we hold the budget fixed and ask which *LoRA variant* a practitioner should pick, and we pair every accuracy claim with a systems measurement on hardware that a single user actually owns. To our knowledge no prior work reports the full model $\times$ method matrix for these six methods with multi-seed statistics and matched consumer-GPU profiling.

3 Background

Let $W_0 \in \mathbb{R}^{d \times k}$ be a frozen pre-trained weight matrix and ΔW the adaptation update. All six methods differ only in how ΔW is parameterized, initialized, or quantized.

LoRA and rsLoRA. LoRA factors the update as $\Delta W = BA$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$:

$$h = W_0 x + \frac{\alpha}{r} B A x. \quad (1)$$

A is Kaiming-initialized and B is zeroed, so $\Delta W = 0$ at step 0. Only A and B receive gradients, $\partial \mathcal{L} / \partial A = B^\top (\partial \mathcal{L} / \partial h) x^\top$ and $\partial \mathcal{L} / \partial B = (\partial \mathcal{L} / \partial h) (A x)^\top$. The α/r scaling attenuates the adapter signal as r grows; rsLoRA (Kalajdziewski, 2023) replaces it with $\alpha/\sqrt{r}$, which keeps adapter activations and gradients from vanishing at large rank.

QLoRA. QLoRA (Dettmers et al., 2023) quantizes W_0 to 4-bit NormalFloat (NF4) and keeps adapters in BF16. NF4 places its 16 levels at equal-probability-mass quantiles of $\mathcal{N}(0, 1)$,

$$q_i = \frac{1}{2} \left[\Phi^{-1} \left(\frac{i-1}{16} \right) + \Phi^{-1} \left(\frac{i}{16} \right) \right], \quad (2)$$

renormalized to $[-1, 1]$ with an exact zero. Each block of 64 weights carries an FP32 scale, costing $\frac{32}{64} = 0.5$ bits/param; double quantization stores those scales as FP8 in super-blocks of 256, cutting the overhead to $\frac{8}{64} + \frac{32}{64 \cdot 256} \approx 0.127$ bits/param, a saving of 0.37 bits/param. Paged optimizers migrate optimizer state between VRAM and host RAM to survive backward-pass memory spikes.

LoftQ. Quantizing W_0 introduces an error that standard QLoRA never sees, because its adapters start at zero. LoftQ (Li et al., 2024) instead solves $\min_{Q(W_0), A, B} \|W_0 - Q(W_0) - BA\|_F^2$ by alternating minimization before training, so the adapter is initialized to absorb the quantization residual.

DoRA. DoRA (Liu et al., 2024) decomposes the weight into magnitude and direction, $W = m V / \|V\|_c$, and applies the low-rank update only to the direction:

$$W = m \frac{W_0 + BA}{\|W_0 + BA\|_c}. \quad (3)$$

The directional gradient carries a projector, $\partial \mathcal{L} / \partial V = \frac{m}{\|V\|_c} (I - V V^\top / \|V\|_c^2) \partial \mathcal{L} / \partial W$, which maps updates onto the tangent space of

the unit sphere and decouples magnitude from direction—the coupling that distinguishes LoRA updates from FFT updates. Expanding the column norm as $\|W_0 + BA\|_{c,i}^2 = \|W_{0,i}\|_2^2 + 2(W_{0,i}^\top B_i)(A_i^\top) + \|B_i A_i\|_2^2$ avoids materializing the dense product, keeping overhead at $O(dr)$ rather than $O(d^2)$.

PiSSA. PiSSA (Meng et al., 2024) keeps LoRA’s parameterization but changes the initialization. Given $W_0 = U\Sigma V^\top$ with $\sigma_1 \geq \sigma_2 \geq \dots$, it splits off the rank- r principal component and initializes

$$B = U_p \Sigma_p^{1/2}, \quad A = \Sigma_p^{1/2} V_p^\top, \quad (4)$$

freezing the residual $W_r = U_r \Sigma_r V_r^\top$. The adapter therefore begins on the dominant singular directions of the pre-trained weight rather than at zero. Crucially for our systems analysis, the trainable topology is *identical* to LoRA’s; only the initial values differ.

AdaLoRA. AdaLoRA (Zhang et al., 2023) parameterizes the update in SVD form and prunes singular directions by an importance score, redistributing a global rank budget across matrices. This is why AdaLoRA’s trainable fraction in Table 2 is lower than the other methods’ at the same nominal rank.

Static memory. For P parameters with adapter fraction $\phi \approx 0.01$, mixed-precision training requires $16P$ bytes for FFT ($2P$ weights, $2P$ gradients, $12P$ AdamW states), $(2 + 16\phi)P \approx 2.16P$ for LoRA, and $(0.516 + 16\phi)P \approx 0.676P$ for 4-bit QLoRA. The model thus predicts a 68.7% VRAM reduction from LoRA to QLoRA; Section 5.5 measures 69.6%. Activation memory, $M_{\text{act}} \approx sb(h + 5sa/d)$ per layer, is handled by gradient checkpointing (Chen et al., 2016), which trades roughly 33% extra FLOPs for a reduction from $O(Lsbh)$ to $O(sbh + L)$.

4 Experimental Setup

All training, evaluation, and profiling code, together with the exact YAML configuration for every run, is released under Apache 2.0 at <https://github.com/KanakMalpani/PEFT>; adapter weights are not released. This section states what a reader needs to reproduce Table 2 without consulting that repository.

Models. Qwen2.5-1.5B-Instruct (1.54B params, 28 layers, $d=1536$, 12 heads) and Qwen2.5-7B-Instruct (7.62B, 28 layers, $d=3584$, 28 heads) (Yang et al., 2024); Llama-3-8B-Instruct (8.03B, 32 layers, $d=4096$, 32 heads) (Grattafiori et al., 2024). All use RoPE (Su et al., 2024) and SwiGLU feed-forward blocks (Shazeer, 2020), and are loaded from the Hugging Face Hub at their default revisions in BF16 (NF4 for QLoRA and LoftQ).

Data. MetaMathQA (Yu et al., 2024), training split, truncated deterministically to its first 100,000 of 395,000 examples; no shuffling, filtering, or deduplication is applied, so the subset is byte-identical across methods, models, and seeds. The release is stored with its eight augmentation types interleaved rather than blocked, so the leading slice spans all of them and no reweighting is introduced by the truncation. Each example is rendered as one ChatML user/assistant turn with no system message (Appendix A), truncated at 2048 tokens, with example packing disabled.

Training. TRL’s SFTTrainer for three epochs. At an effective batch of 128 this is 781 optimizer steps per epoch and 2,343 in total. AdamW at 2×10^{-4} on a cosine schedule with 3% warmup; remaining optimizer settings are the Hugging Face TrainingArguments defaults ($\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 0, gradient-norm clipping at 1.0). Gradient checkpointing is enabled throughout. The effective batch of 128 is reached as micro-batch 2×64 accumulation steps on the 1.5B model and micro-batch 1×128 on the 7B and 8B models; this is the only setting that differs across models.

Adapters. $r=16$, $\alpha=32$, dropout 0.05, no bias term, targeting $W_q, W_k, W_v, W_o, W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$. PiSSA and LoftQ are applied through peft’s adapter initialization (LoftQ at 4 bits), DoRA through its weight-decomposition flag, and AdaLoRA with initial rank 16 pruned toward a target rank of 8 ($t_{\text{init}}=200$, $t_{\text{final}}=1000$, $\Delta T=10$) — which is why its trainable fraction in Table 2 is lower than the others’ at the same nominal rank. Quantized runs use NF4 with double quantization and a BF16 compute dtype.

Evaluation. GSM8K (Cobbe et al., 2021) (main test split, 1,319 problems) and MATH (Hendrycks et al., 2021) (competition-math test split), both zero-shot with greedy decoding and at most 512

new tokens. Scoring is exact match on a normalized final answer: the last `\boxed{ }` expression if present, otherwise the value following `####`, otherwise the last numeric token of the last non-empty line; commas, currency, and percent signs are stripped and integral values are cast to integers before comparison.

Training budget. Every configuration in this paper uses the *same* hyperparameters (Table 1); no per-method tuning was performed. This measures how the methods behave under the single budget a practitioner is most likely to adopt—the PEFT library defaults—not how they behave at their individual optima, a limitation Section LIMITATIONS returns to. The ablations of Section 5.4 run the identical pipeline with only the corresponding configuration key overridden.

Software and hardware. Ubuntu 22.04, PyTorch 2.2.1, CUDA 12.1, transformers 4.40, peft 0.10, trl 0.8, bitsandbytes 0.43, datasets 2.18, accelerate 0.28, FlashAttention-2 (Dao, 2024). Training and profiling on an RTX 4060 (8 GB) for 1.5B and an RTX 4090 (24 GB) for 7B/8B.

Profiling protocol. VRAM and throughput are measured on a synthetic fixed-length batch at the micro-batch size and 2048 tokens rather than on real data, so that the measurement isolates the adaptation method from sequence-length variation: 50 warm-up steps followed by 500 measured forward–backward–step iterations, with peak VRAM read from the CUDA allocator’s high-water mark. Energy is an *estimate* rather than a rail measurement: rated GPU board power multiplied by wall-clock time, at 115 W for the RTX 4060 and 450 W for the RTX 4090. It therefore excludes CPU, memory, and power-supply overhead, and is best read as a comparison between rows on the same card rather than as an absolute figure. The PagedAdamW rows substitute the paged 32-bit AdamW implementation for the standard one, holding everything else fixed.

Seeds. Three seeds (42, 123, 456) per configuration; we report mean $\pm$ standard deviation throughout. Because decoding is greedy, evaluation contributes no additional variance.

Setting	Value
Base precision	BF16 (NF4 for QLoRA/LoftQ)
Learning rate	2×10^{-4}
LR scheduler	cosine, 3% warmup
Optimizer	AdamW, $\beta = (0.9, 0.999)$ $\epsilon=10^{-8}$, wd 0
Grad. clipping	1.0
Epochs	3 (2,343 optimizer steps)
Rank r / α / dropout	16 / 32 / 0.05
Target modules	all linear (attn. + FFN)
Effective batch	128 [†]
Sequence length	2048 (packing off)
Grad. checkpointing	enabled
Seeds	42, 123, 456
Decoding (eval)	greedy, ≤ 512 new tokens

Table 1: Training configuration, held fixed across every model and PEFT method. [†]Reached as micro-batch 2×64 accumulation steps on Qwen2.5-1.5B and 1×128 on Qwen2.5-7B and Llama-3-8B; this is the only setting that varies across models.

5 Results

5.1 Accuracy across the full matrix

Table 2 reports every model $\times$ method cell. Three observations organize the rest of this section.

A 1.5B model overtakes an unadapted 7B model.

Qwen2.5-1.5B + PiSSA reaches $71.40 \pm 0.27\%$ on GSM8K against $68.50 \pm 0.42\%$ for the unadapted Qwen2.5-7B: a +2.90 point gap with Welch’s $t(3.41) = 10.06$, $p = 0.0012$, Cohen’s $d = 8.21$. This is the headline practical result—for a task that is well specified in advance, a 1% adapter on a 1.5B backbone buys more than $4.9 \times$ the parameters.

The method ordering is scale-invariant. Ranking the six methods by GSM8K accuracy gives PiSSA > DoRA > LoRA > AdaLoRA > LoftQ > QLoRA on Qwen2.5-1.5B, and *the same ordering*, position for position, on Qwen2.5-7B and Llama-3-8B (Kendall’s $\tau = 1.0$ for all three pairwise comparisons; the ordering on MATH is identical as well). The reviewers’ question of whether the ranking generalizes across scale and model family therefore has a clean answer in this regime: it does. What changes with scale is the *size* of the adaptation itself, not its ordering—the PiSSA–LoRA gap is +2.20, +2.10, and +2.20 points at 1.5B, 7B, and 8B respectively.

MATH gains are larger in relative terms. Absolute MATH accuracy is low, which is expected for models of this size on competition-level problems, but the relative effect of adaptation is consistently *larger* on MATH than on GSM8K: +88.7%

Configuration	Tr. (%)	GSM8K (%)	MATH (%)
<i>Qwen2.5-1.5B-Instruct</i>			
unadapted	0.00	52.10 $\pm$ 0.42	14.20 $\pm$ 0.57
+ QLoRA	1.12	68.40 $\pm$ 0.21	23.90 $\pm$ 0.44
+ LoftQ	1.12	68.90 $\pm$ 0.31	24.40 $\pm$ 0.41
+ AdaLoRA	0.84	69.05 $\pm$ 0.15	24.60 $\pm$ 0.38
+ LoRA	1.12	69.20 $\pm$ 0.12	24.80 $\pm$ 0.35
+ DoRA	1.13	70.80 $\pm$ 0.18	26.10 $\pm$ 0.30
+ PiSSA	1.12	71.40 $\pm$ 0.27	26.80 $\pm$ 0.22
<i>Qwen2.5-7B-Instruct</i>			
unadapted	0.00	68.50 $\pm$ 0.42	22.40 $\pm$ 0.51
+ QLoRA	0.84	80.50 $\pm$ 0.14	37.50 $\pm$ 0.31
+ LoftQ	0.84	81.00 $\pm$ 0.11	38.00 $\pm$ 0.28
+ AdaLoRA	0.62	81.10 $\pm$ 0.13	38.30 $\pm$ 0.24
+ LoRA	0.84	81.20 $\pm$ 0.09	38.40 $\pm$ 0.25
+ DoRA	0.85	82.70 $\pm$ 0.10	40.20 $\pm$ 0.19
+ PiSSA	0.84	83.30 $\pm$ 0.08	41.10 $\pm$ 0.15
<i>Llama-3-8B-Instruct</i>			
unadapted	0.00	70.20 $\pm$ 0.38	23.10 $\pm$ 0.49
+ QLoRA	0.78	79.10 $\pm$ 0.15	35.40 $\pm$ 0.33
+ LoftQ	0.78	79.60 $\pm$ 0.12	35.90 $\pm$ 0.30
+ AdaLoRA	0.58	79.70 $\pm$ 0.14	36.00 $\pm$ 0.27
+ LoRA	0.78	79.80 $\pm$ 0.11	36.20 $\pm$ 0.29
+ DoRA	0.79	81.40 $\pm$ 0.10	38.10 $\pm$ 0.21
+ PiSSA	0.78	82.00 $\pm$ 0.07	38.90 $\pm$ 0.18

Table 2: Full model $\times$ method matrix: exact-match accuracy, mean $\pm$ std over three seeds. “Tr.” is the trainable-parameter fraction. Rows are ordered by GSM8K accuracy within each block; the ordering is identical in all three blocks and on both benchmarks.

vs. +37.0% relative for Qwen2.5-1.5B (base $\rightarrow$ PiSSA), +83.5% vs. +21.6% for Qwen2.5-7B, and +68.4% vs. +16.8% for Llama-3-8B. The harder benchmark is where the unadapted models are furthest from the target output format and solution style, and therefore where the adapter has the most to correct. The ordering of methods is unchanged, so MATH does not alter the practical recommendation—it strengthens it.

5.2 Which differences are real?

Standard deviations alone do not tell a practitioner whether a 0.3-point gap is worth acting on. We therefore run Welch’s t -test on each method pair from the three-seed statistics and apply Holm–Bonferroni correction across the 14 method-versus-method tests (Table 3).

The result is a clean two-tier picture. PiSSA and DoRA beat LoRA by margins that survive correction on GSM8K ($p_{\text{Holm}} = 0.020$ and 0.007), and PiSSA does so on MATH as well ($p_{\text{Holm}} = 0.027$). Everything else collapses: QLoRA, LoftQ, and AdaLoRA are *not* distinguishable from LoRA after correction on either benchmark, and neither is LoftQ from QLoRA. Even PiSSA versus DoRA—a

Comparison	Δ	t	p	p_{Holm}
<i>GSM8K</i>				
PiSSA – LoRA	+2.20	12.90	.002	.020
DoRA – LoRA	+1.60	12.81	.001	.007
PiSSA – DoRA	+0.60	3.20	.040	.319
LoRA – AdaLoRA	+0.15	1.35	.251	1.00
LoRA – LoftQ	+0.30	1.56	.230	1.00
LoRA – QLoRA	+0.80	5.73	.009	.095
LoftQ – QLoRA	+0.50	2.31	.091	.545
<i>MATH</i>				
PiSSA – LoRA	+2.00	8.38	.002	.027
DoRA – LoRA	+1.30	4.88	.009	.095
PiSSA – DoRA	+0.70	3.26	.035	.318
LoRA – AdaLoRA	+0.20	0.67	.540	1.00
LoRA – LoftQ	+0.40	1.29	.270	1.00
LoRA – QLoRA	+0.90	2.77	.053	.371
LoftQ – QLoRA	+0.50	1.44	.224	1.00
<i>Headline claim (GSM8K)</i>				
1.5B PiSSA – 7B base	+2.90	10.06	.001	—

Table 3: Welch’s t -tests on Qwen2.5-1.5B ($n=3$ seeds per arm), Holm–Bonferroni corrected across the 14 method-pair tests. Bold marks $p_{\text{Holm}} < 0.05$. The headline row (Cohen’s $d = 8.21$) is excluded from the correction, as it is not a method-versus-method comparison.

+0.60 point gap that looks decisive against a ± 0.27 standard deviation—does not survive ($p_{\text{Holm}} = 0.319$).

This has a direct practical consequence, and it is the opposite of what a ranked table invites the reader to conclude. The choice between LoRA, QLoRA, LoftQ, and AdaLoRA should be made on *memory and engineering grounds* (Section 5.5), because at $n=3$ their accuracy differences are noise. The choice to use PiSSA or DoRA over LoRA is supported by the data; the choice between PiSSA and DoRA is not.

5.3 Why PiSSA wins

The significance analysis establishes *that* PiSSA and DoRA separate from LoRA. The size of the PiSSA effect says something about the mechanism.

Cross-referencing Table 2 against the rank sweep of Section 5.4 gives the sharpest statement of it: PiSSA at $r=16$ scores $71.40 \pm 0.27\%$ on GSM8K, while LoRA needs $r=64$ to reach 70.90% and still does not catch it at $r=128$ (71.00%). *Spectral initialization at rank 16 is worth more than quadrupling the rank budget*, at a quarter of the adapter parameters and none of the extra memory. Under a fixed parameter budget, where the adapter goes and how it starts both dominate how large it is.

The mechanism is initialization, not capacity.

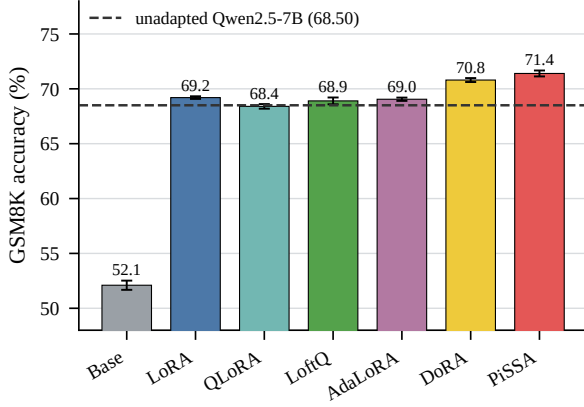


Figure 1: GSM8K accuracy on Qwen2.5-1.5B (mean $\pm$ std, three seeds). The dashed line is the unadapted Qwen2.5-7B baseline. Only the PiSSA and DoRA margins over LoRA survive multiple-comparison correction (Table 3).

LoRA begins at $\Delta W = 0$, with A random and B zeroed, and must first discover a useful update direction; PiSSA begins at $B = U_p \Sigma_p^{1/2}$, $A = \Sigma_p^{1/2} V_p^\top$ —already aligned with the dominant singular directions of the pre-trained weight, with the residual W_r frozen (Meng et al., 2024). With the rank budget fixed, the directions LoRA spends its early updates searching for are directions PiSSA is handed at initialization, and a rank-16 subspace chosen by the SVD is a better use of 16 directions than a rank-64 subspace found by gradient descent from noise.

DoRA improves on LoRA for a different reason: magnitude–direction decoupling in the parameterization rather than a better starting point (Liu et al., 2024). It reaches a comparable end point ($70.80 \pm 0.18\%$). That the two strongest methods improve on LoRA through different mechanisms is a plausible reason why their margins over LoRA are significant while their margin over each other is not (Table 3): they arrive at similar places by different routes. Separating them would need either more seeds or a budget at which the two mechanisms compound, and we test neither.

5.4 Ablations

Rank. On Qwen2.5-1.5B with LoRA, GSM8K accuracy rises from 67.2% at $r=8$ to 69.2% ($r=16$), 70.4% ($r=32$), 70.9% ($r=64$), and 71.0% ($r=128$) (Figure 2a). The last doubling buys 0.1 points for twice the adapter parameters; $r=64$ is the practical ceiling.

Target modules. Placement matters more than rank (Figure 2b). Adapting only W_q, W_v gives $63.40 \pm 0.28\%$ GSM8K; all four attention projections give $66.10 \pm 0.21\%$; all linear layers including the FFN give $69.20 \pm 0.12\%$. The +5.8-point span dwarfs the differences between adaptation *methods* at fixed placement, which is the single most actionable finding in this paper: a practitioner who targets attention only loses more than any choice of method can recover.

Adapter scaling. Standard LoRA is non-monotonic in rank on MATH— $25.20 \pm 0.29\%$ at $r=8$, $27.10 \pm 0.24\%$ at $r=16$, but $25.10 \pm 0.40\%$ at $r=64$ —even though GSM8K accuracy rises monotonically over the same sweep. This is the signal attenuation that Kalajdzievski (2023) attributes to the α/r factor, whose effect grows with rank: the two scalings differ by a factor of $\sqrt{r}$ independent of α , so eightfold at $r=64$ against fourfold at $r=16$. Substituting rank-stabilized scaling at $r=64$ recovers the loss and then some: $28.30 \pm 0.18\%$, a 3.2-point gain over standard scaling at the same rank and 1.2 points above the best standard-scaling result at any rank. We ran the substitution at the rank where the two scalings diverge most; everywhere else in this paper we use standard α/r scaling at $r=16$, so this ablation does not bear on our other results.

5.5 Systems profiling

Table 4 reports peak VRAM, throughput, and energy for every configuration we profiled, and Figure 3 plots the trade-off.

Quantization. For Qwen2.5-7B, moving from BF16 LoRA to 4-bit QLoRA takes peak VRAM from 22.4 GB to 6.8 GB, a 69.6% reduction; adding PagedAdamW brings it to 6.1 GB, or 72.8%. The static-memory model of Section 3 predicts 68.7%, so the analytic estimate is accurate to within a point once activation memory and checkpointing are accounted for. The practical consequence is a hardware-class change: 7B adaptation moves from a 24 GB card to inside the 8 GB envelope of an RTX 4060.

Throughput. Quantization costs 21–24% throughput across scales ($612 \rightarrow 485$ tok/s at 1.5B; $342 \rightarrow 261$ at 7B; $315 \rightarrow 242$ at 8B), because NF4 weights must be dequantized to BF16 before each matmul. PagedAdamW costs a further 15–20% ($485 \rightarrow 412$ and $261 \rightarrow 210$) from PCIe transfer of optimizer state. Energy per 10k tokens moves the

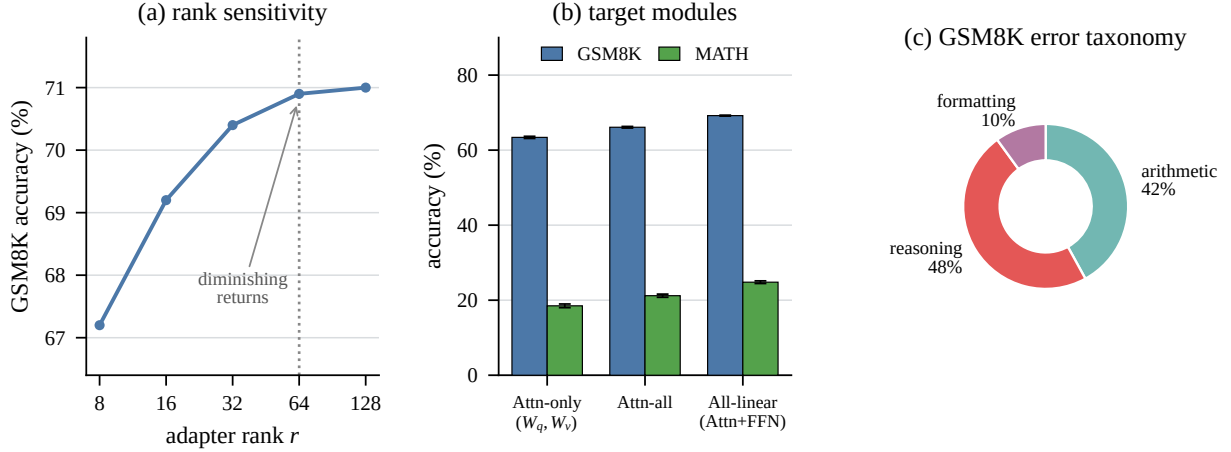


Figure 2: Ablations and failure analysis on Qwen2.5-1.5B. (a) GSM8K accuracy against adapter rank with LoRA; gains flatten beyond $r=64$. (b) Adapter placement: extending from attention-only to all linear layers is worth +5.8 GSM8K and +6.3 MATH points. (c) Taxonomy of 100 audited GSM8K failures.

opposite way to VRAM: because the estimate is board power times wall-clock time, the 4-bit paths are slower and therefore *less* energy-efficient per token despite the smaller footprint—26% more energy for 1.5B QLoRA than LoRA, 31% more for 7B. This is worth stating plainly, because “quantization saves resources” is usually read as saving all of them; here it buys memory with time and energy.

What these rates are. Table 4 profiles a fixed-length batch at 2048 tokens, which is the rate the hardware sustains when it is fully occupied. Our training batches are variable-length with packing disabled, so a micro-batch carries a few hundred tokens rather than 2×2048 and utilization is far lower; the table therefore reports *achievable* throughput per method and card, which is what a practitioner choosing between methods needs, rather than the rate our own runs achieved. Sequence packing would close most of that gap and is the first thing we would change about this setup.

On the absent methods. Table 4 profiles LoRA and QLoRA rather than all six methods. PiSSA’s trainable topology is identical to LoRA’s—same B and A shapes, same forward and backward computation, only the initial values differ—so its steady-state VRAM and throughput are LoRA’s by construction, its one-time SVD cost being paid before step 0; LoftQ stands in the same relation to QLoRA, and AdaLoRA trains *fewer* parameters at the same nominal rank (Table 2), so it is bounded above by the LoRA row. DoRA is the genuine gap: its magnitude vector and column-norm computation add

Model / GPU	Method (opt.)	VRAM (GB)	tok/s	Wh per 10k tok
1.5B / 4060	LoRA (AdamW)	6.2	612	0.52
1.5B / 4060	QLoRA (AdamW)	3.4	485	0.66
1.5B / 4060	QLoRA (Paged)	3.1	412	0.78
7B / 4090	LoRA (AdamW)	22.4	342	3.65
7B / 4090	QLoRA (AdamW)	6.8	261	4.78
7B / 4090	QLoRA (Paged)	6.1	210	5.94
8B / 4090	LoRA (AdamW)	24.1	315	3.96
8B / 4090	QLoRA (AdamW)	7.6	242	5.16

Table 4: Consumer-GPU profiling. “Paged” is PagedAdamW. Energy is rated GPU board power $\times$ wall-clock time (115 W for the 4060, 450 W for the 4090), so it is comparable within a card but not across cards. Every row plotted in Figure 3 appears here.

overhead that our $O(dr)$ analysis bounds but that we did not measure, which we flag in Section LIMITATIONS rather than estimate.

5.6 Failure analysis

Protocol. We sampled 100 incorrect GSM8K generations uniformly at random from the Qwen2.5-1.5B + LoRA seed-1 run and assigned each to exactly one of three pre-registered categories: *arithmetic* (correct equation set-up, wrong calculation), *reasoning* (broken or skipped chain-of-thought step, wrong quantity used), and *formatting* (ChatML template violation or degenerate repetition). Where a generation contained both an arithmetic and a reasoning error, the earliest error in the chain was taken as the label. Annotation was performed by a single annotator (the author), so no inter-annotator agreement statistic is available; this

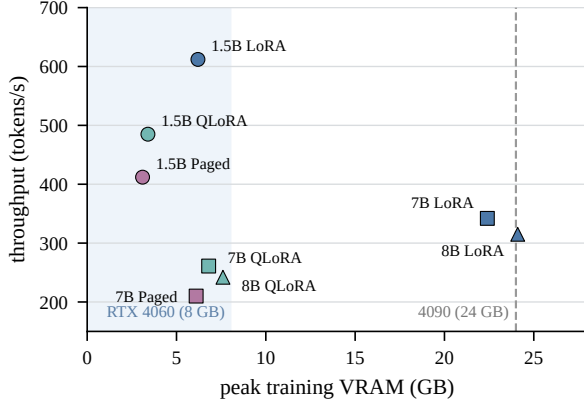


Figure 3: Peak VRAM against training throughput. The shaded region is the 8 GB RTX 4060 envelope: 4-bit quantization moves 7B and 8B adaptation inside it at a 21–24% throughput cost.

is a limitation of the analysis rather than a property of the taxonomy.

Findings. Reasoning failures dominate at 48%, arithmetic at 42%, formatting at 10% (Figure 2c). Two examples illustrate the distinction. *Arithmetic*: given “Sarah has 14 apples and buys 3 boxes of 8,” the model sets up $14 + 3 \times 8$ correctly but evaluates $3 \times 8 = 22$, yielding 36 instead of 38. *Reasoning*: on a pool fill/drain problem the model computes the net rate correctly and then divides by the wrong quantity.

The 42% arithmetic share is the more actionable half: it is a failure mode that tool use or a calculator-augmented decoding pass addresses directly, whereas the 48% reasoning share is not obviously fixable by adaptation at this scale. Formatting errors, at 10%, are almost entirely eliminated by fine-tuning relative to the unadapted baseline—which is precisely the capability adapters are best at supplying.

6 Discussion

Why adapters suffice for small models. Domain adaptation on a task like MetaMathQA is largely *behavioral*: the base model already contains the arithmetic and algebraic knowledge, and what fine-tuning supplies is a solution format and a reasoning style. Adapters modify behavior without the parameter capacity needed to store new world knowledge, which is why a 1% adapter closes—and here reverses—a $4.9\times$ parameter gap. Constraining the update to a low-rank subspace also regularizes, consistent with the low intrinsic dimension of fine-tuning updates (Aghajanyan et al.,

2021) and the reduced forgetting Biderman et al. (2024) report.

Serving. Because the base weights stay frozen, one backbone can serve many adapters. Punica (Chen et al., 2024) batches heterogeneous adapters with a segmented gather matrix–vector kernel; S-LoRA (Sheng et al., 2024) pages an adapter catalog between host and device memory, which scales the catalog at the cost of variable latency and forfeited CUDA graph capture; static hot-set layouts trade catalog size for determinism. Rank-heterogeneous co-batching is the practical failure mode—small adapters inherit the latency of large ones—and rank-aware placement addresses it (Jaiswal et al., 2025). Our finding that $r=64$ is the accuracy ceiling is directly relevant here: capping rank at 64 keeps a catalog rank-homogeneous at almost no accuracy cost.

Reinforcement learning. GRPO (Shao et al., 2024) generates multiple completions per prompt and requires a frozen reference policy alongside the trainable one, splitting VRAM between an inference engine (KV cache, Kwon et al., 2023) and a training engine. Under the 8 GB budget profiled here, 4-bit adapters are what make the reference-policy copy affordable at all.

Recommendations. Under a hard VRAM limit, use QLoRA and accept the 21–24% throughput cost; add PagedAdamW only if you are actually hitting OOM, since it costs a further 15–20% for VRAM you may not need. When accuracy is the objective and memory is not binding, use PiSSA (or DoRA) and target all linear layers—placement is worth more than method. At $r > 32$, use rank-stabilized scaling. Do not choose between LoRA, QLoRA, LoftQ, and AdaLoRA on the basis of accuracy: at this sample size they are indistinguishable (Table 3), so choose on memory.

7 Conclusion

Under a single fixed budget, across three base models and two mathematical reasoning benchmarks, PEFT-adapted small language models match or exceed larger unadapted ones: Qwen2.5-1.5B + PiSSA beats unadapted Qwen2.5-7B on GSM8K by 2.90 points ($p = 0.001$). The method ordering is identical at every scale we tested, but multiple-comparison correction shows that only the PiSSA/DoRA advantage over LoRA is statistically real—the LoRA/QLoRA/LoftQ/AdaLoRA

cluster should be separated on memory grounds, not accuracy. Target-module placement outweighs method choice, 4-bit quantization buys a hardware-class change for a 21–24% throughput cost, and reasoning rather than arithmetic errors dominate the residual failures.

Limitations

Single domain. Every result is trained on MetaMathQA and evaluated on GSM8K and MATH. Whether the method ordering transfers to code, dialogue, or retrieval-augmented tasks is untested, and we make no claim that it does.

Reliability is narrowly scoped. We use “reliable” to mean multi-seed measurement and characterized failure modes (Section 1). We do not test robustness to symbolic perturbation (Mirzadeh et al., 2025), nor whether mathematical adaptation degrades general capability on benchmarks such as MMLU-Pro (Wang et al., 2024) or ARC-Challenge (Clark et al., 2018). Those evaluations are the natural next step and would test the reliability claim in its stronger sense.

One budget, not per-method optima. All methods share one rank, α , learning rate, and module set. This measures the practitioner-default regime rather than each method’s best achievable accuracy, and methods with different optimal hyperparameters—plausibly AdaLoRA, whose rank budget interacts with the schedule, and LoftQ, whose alternating initialization interacts with learning rate—may be disadvantaged. Our conclusions should be read as conditional on this budget.

Three seeds. $n = 3$ gives Welch tests roughly 2–4 degrees of freedom, so the analysis in Section 5.2 is well powered only for large effects. The correct reading of a non-significant row is “not separable at this sample size,” not “equal.”

Incomplete systems matrix. DoRA’s runtime overhead is bounded analytically ($O(dr)$) but not measured. PiSSA, LoftQ, and AdaLoRA inherit their profiles from LoRA/QLoRA by construction, as argued in Section 5.5, but this is an argument rather than a measurement. Energy figures are device-power-times-wall-clock estimates, not metered measurements, and are comparable between rows rather than absolute.

Single annotator. The error taxonomy has no inter-annotator agreement statistic, and its 100-item

sample gives roughly ± 10 points of binomial uncertainty per category.

Ethics Statement

This work fine-tunes and evaluates publicly released models (Qwen2.5, Llama-3) on publicly released datasets (MetaMathQA, GSM8K, MATH), each used in accordance with its license. No human subjects, personal data, or crowdsourced annotation were involved; the error taxonomy was annotated by the author. The paper’s efficiency findings lower the compute cost of specializing language models, which broadens access but equally lowers the cost of producing narrowly capable models without oversight. We report a per-configuration energy estimate (Section 4) to make the environmental cost of adaptation explicit rather than implicit; the 54 training runs behind Table 2, each making 300,000 example passes on a single consumer GPU, carry a substantial cost of their own.

Acknowledgments

The author thanks his parents for their unwavering support and encouragement throughout this work, and thanks the anonymous reviewers of GroundLM 2026, whose requests for the completed model $\times$ method matrix, statistical significance testing, expanded method descriptions, and fuller reproducibility detail shaped this version of the paper.

References

- Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. 2021. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*. ArXiv:2012.13255.
- Dan Biderman, Jacob Portes, Jose Javier Gonzalez Ortiz, Mansheej Paul, Philip Greengard, Connor Jennings, Daniel King, Sam Havens, Vitaliy Chiley, Jonathan Frankle, Cody Blakeney, and John P. Cunningham. 2024. LoRA learns less and forgets less. *Transactions on Machine Learning Research*. ArXiv:2405.09673.
- Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2024. Punica: Multi-tenant LoRA serving. In *Proceedings of Machine Learning and Systems (MLSys)*. ArXiv:2310.18547.
- Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*.

- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Tri Dao. 2024. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*. ArXiv:2307.08691.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient fine-tuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2305.14314.
- Aaron Grattafiori et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. ArXiv:2103.03874.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning (ICML)*. ArXiv:1902.00751.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2106.09685.
- Shashwat Jaiswal, Shrikara Arun, Anjaly Parayil, Ankur Mallick, Spyros Mastorakis, Alind Khare, Chloi Alverti, Renee St Amant, Chetan Bansal, Victor Rühle, and Josep Torrellas. 2025. Serving heterogeneous LoRA adapters in distributed LLM inference systems. *arXiv preprint arXiv:2511.22880*.
- Damjan Kalajdzievski. 2023. A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint arXiv:2312.03732*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. ArXiv:2309.06180.
- Yixiao Li, Yifan Yu, Chen Liang, Pengcheng He, Nikos Karampatziakis, Weizhu Chen, and Tuo Zhao. 2024. LoftQ: LoRA-fine-tuning-aware quantization for large language models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2310.08659.
- Vladislav Lialin, Vijeta Deshpande, Xiaowei Yao, and Anna Rumshisky. 2023. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.15647*.
- Shih-Yang Liu, Chien-Yi Wang, Hongxu Yin, Pavlo Molchanov, Yu-Chiang Frank Wang, Kwang-Ting Cheng, and Min-Hung Chen. 2024. DoRA: Weight-decomposed low-rank adaptation. In *International Conference on Machine Learning (ICML)*. ArXiv:2402.09353.
- Fanxu Meng, Zhaohui Wang, and Muhan Zhang. 2024. PiSSA: Principal singular values and singular vectors adaptation of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*. ArXiv:2404.02948.
- Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. 2025. GSM-symbolic: Understanding the limitations of mathematical reasoning in large language models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.05229.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Noam Shazeer. 2020. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*.
- Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2024. S-LoRA: Serving thousands of concurrent LoRA adapters. In *Proceedings of Machine Learning and Systems (MLSys)*. ArXiv:2311.03285.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2024. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063. ArXiv:2104.09864.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. 2024. MMLU-pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. ArXiv:2406.01574.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhengguo Li, Adrian Weller, and Weiyang Liu. 2024. MetaMath: Bootstrap your own mathematical questions for large language models. In *International Conference on Learning Representations (ICLR)*. ArXiv:2309.12284.

Xin Yu, Cong Xie, Xunmei Liu, Tiantian Fan, Lingzhou Xue, and Zhi Zhang. 2025. Train large, deploy compact: Structured compression for compact low-rank adaptation. *arXiv preprint arXiv:2510.00192*.

Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. 2023. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations (ICLR)*. ArXiv:2303.10512.

A Prompt Template

Training examples are rendered as a single ChatML user/assistant turn with no system message and no in-context exemplars:

```
<|im_start|>user
{query}
<|im_start|>assistant
{response}
```

At evaluation time the prompt is truncated after the assistant marker and the model generates the continuation:

```
<|im_start|>user
{question}
<|im_start|>assistant
```

For faithfulness we note that this template opens each turn with `<|im_start|>` but emits no closing `<|im_end|>` terminator. Scoring is unaffected—generation is bounded by the tokenizer’s EOS token and the 512-token cap, and only the final answer is read—but the absence of an explicit turn terminator is a plausible contributor to the degenerate-repetition cases counted under *formatting* in Section 5.6.

B Reproduction Checklist

- **Code.** <https://github.com/KanakMalpani/PEFT> (Apache 2.0). Training, evaluation, and profiling entry points with per-method and per-model YAML configurations. Adapter weights are not released.
- **Base models.** Qwen/Qwen2.5-1.5B-Instruct, Qwen/Qwen2.5-7B-Instruct, meta-llama/Meta-Llama-3-8B-Instruct, loaded at their default Hub revisions (no revision pin).

- **Data.** meta-math/MetaMathQA, train split, first 100,000 examples taken deterministically; no shuffling, filtering, or deduplication.
- **Training.** TRL SFTTrainer, 3 epochs = 2,343 optimizer steps at effective batch 128, sequence length 2048, packing disabled, gradient checkpointing enabled, BF16.
- **Adapters.** $r=16$, $\alpha=32$, dropout 0.05, bias none, targets $W_q, W_k, W_v, W_o, W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$. AdaLoRA: init rank 16, target rank 8, $t_{\text{init}}=200$, $t_{\text{final}}=1000$, $\Delta T=10$. LoftQ: 4 bits. Quantization: NF4, double quantization, BF16 compute dtype.
- **Optimizer.** AdamW, lr 2×10^{-4} , cosine schedule, 3% warmup, $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$, weight decay 0, gradient-norm clipping 1.0.
- **Evaluation.** gsm8k (main, test) and hendrycks/competition_math (test); zero-shot, greedy decoding, ≤ 512 new tokens; exact match on the normalized final answer (§4).
- **Seeds.** 42, 123, 456. Evaluation is deterministic under greedy decoding.
- **Hardware and scale.** RTX 4060 8 GB (1.5B), RTX 4090 24 GB (7B/8B). Table 2 rests on 54 training runs (three models $\times$ six methods $\times$ three seeds), plus the rank, target-module and scaling sweeps of Section 5.4. Each run makes $3 \times 100,000$ example passes, so the grid is 1.62×10^7 example passes and 1.35×10^7 forward-backward steps; every resulting checkpoint is then evaluated on 6,319 GSM8K and MATH problems with batch-1 greedy decoding. We do not report a single wall-clock total, because the throughputs in Table 4 are measured at full sequence length and overstate the rate achieved on variable-length training batches (Section 5.5).
- **Profiling.** Synthetic fixed-length batch at the micro-batch size and 2048 tokens; 50 warm-up and 500 measured steps; peak VRAM from the CUDA allocator high-water mark; energy estimated as assumed device power (450 W / 200 W) $\times$ wall-clock time.
- **Stack.** PyTorch 2.2.1, CUDA 12.1, transformers 4.40, peft 0.10, trl 0.8, bitsandbytes 0.43, datasets 2.18, accelerate 0.28, FlashAttention-2.